MezFlow: A Real-Time Restaurant Dining and Kitchen Operations Management System

Sheikh Muhammad Khizar (2380227)

Talha Mumtaz (2380700)

Syed Sail Haider Gardezi (2380245)

Supervised by: Muhammad Farooq

In partial fulfillment of requirement for the degree
Bachelor of Science (Software Engineering)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
Islamabad, Pakistan

Fall, 2026

MezFlow: A Real-Time Restaurant Dining and Kitchen Operations Management System

By

Sheikh Muhammad Khizar (2380227), Talha Mumtaz (2380700), Syed Sail Haider Gardezi (2380245)

CERTIFICATE

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING (BSSE).

It is to certify that the above students thesis has been completed to my satisfaction and, to my belief, its standard is appropriate for submission for Evaluation. I have also conducted plagiarism test of this thesis using HEC prescribed software and found a similarity index of _____% that is within the permissible limit set by the HEC for the BSSE degree thesis. I have also found the thesis in a format recognized by the SZABIST for the BSSE thesis.

_____
(Supervisor)
Muhammad Farooq

_____
(FYP Committee Head)
_____

Date: _____

Department of Computer Science, Shaheed Zulfikar Ali Bhutto Institute of Science and Technology, Islamabad Campus

Fall, 2026

Project Overview

Restaurants depend on close coordination between customers, waiters, kitchen staff, cashiers and managers, yet many small and medium dine-in restaurants still rely on printed menus, verbal ordering and manual kitchen coordination. Existing digital tools usually stop at showing a menu or printing a kitchen ticket. They rarely let several diners order together, give the kitchen one organised view of every active order, flag delayed dishes automatically, or tell a paid bill apart from a table that has actually been cleaned. MezFlow is proposed as a real-time restaurant dining and kitchen operations management system that connects these workflows in one platform. Diners scan a code at their table and join one shared order in their web browser without installing an application. A single-screen kitchen display shows every active order with status and category filters, and the system tracks preparation time and flags delays. A management console configures menus, tables and staff, and an artificial intelligence feature explains dishes in English and Urdu using only information supplied by the restaurant. Orders and tables follow two independent lifecycles, so payment closes the digital session but a table becomes available again only after staff confirm that it has been cleaned. The system is organised into twelve modules and is developed with the Agile Scrum method in two-week sprints using open-source technologies. It is compared with ten existing restaurant technology products, and it will be evaluated through load testing for real-time response and concurrent use, automated tests for correctness, a review of whether dish explanations stay within the restaurant's own information, and a usability study with at least ten participants aiming for a usability score of at least seventy. At this stage the project is a proposal and plan, development proceeds over two semesters, and future work includes additional restaurant locations, languages and payment methods.

Dedication

Firstly, we dedicate our project to the creator Allah Almighty and dedicate to whom the world owes its existence Muhammad (Peace Be Upon Him) and dedicate this to our beloved parents, our extremely dedicated and generous teachers and supportive friends, their prayers always pave the way to success for us.

Acknowledgement

We are grateful to Allah Almighty for giving us the ability and the opportunity to undertake this project. We sincerely thank our supervisor, Muhammad Farooq, for his guidance, feedback and encouragement throughout the work on this project, and we thank the faculty of the Department of Computer Science at the Shaheed Zulfikar Ali Bhutto Institute of Science and Technology, Islamabad Campus, for their support. We also thank our families and friends for their patience and prayers. This work is submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering.

_____	_____	_____
(Student 1)	(Student 2)	(Student 3)
Sheikh Muhammad Khizar	Talha Mumtaz	Syed Sail Haider Gardezi

Revision History

Compiled By	Checked By	Date	Reason for Change	Version
Sheikh Muhammad Khizar		21st Oct 2026	Initial Version: front matter and Chapter 1 (Introduction)	1.0

Weekly record of changes agreed with the supervisor.

Contents

Project Overview i
Dedication ii
Acknowledgements iii
Revision History iv
Contents v
List of Figures vi
List of Tablesv ii
1 Introduction 1
1.1 Product Purpose 1
1.2 Product Scope 2
1.2.1 Existing System Description 2
1.2.2 Future System Usage Analysis 7
1.3 Objectives 8
1.4 Problem Statement / Limitations 9
1.5 Proposed Solution 10
1.6 Intended Market of Product 11
1.7 Intended Users of Product 11
1.8 Software Process Model 12
1.8.1 Process Model Introduction 12
1.8.2 Justification of Proposing the Process Model 13
1.8.3 Steps of Process Model 13
References 16

List of Figures

1.1 Agile process model 14

List of Tables

1.1 Applications Comparison 8

Chapter 1: Introduction

MezFlow is a real-time restaurant dining and kitchen operations management system that connects the customer, kitchen, billing and management workflows of a small or medium dine-in restaurant through a single platform. A point-of-sale (POS) system records customer orders and takes payment, and vendors such as Toast and Square bundle ordering, payment processing and a kitchen display with it [1, 2]. MezFlow keeps that idea but extends it to the whole dining experience, from the moment diners sit down until the table is ready for the next guests.

A Quick Response (QR) code is a two-dimensional barcode that a phone camera can read and open as a web address [3]. In MezFlow every table carries a QR code that opens the digital menu in the diner's browser as a Progressive Web App (PWA), so nothing has to be installed [4]. Several diners who scan the same code join one shared table session, add items to one live cart and see each other's changes, while the system still records which diner added each item. Structured modifiers such as spice level, size, extras and removals travel with every item, so the kitchen does not depend on verbal instructions [5].

A kitchen display system (KDS) replaces paper tickets with a screen that shows the orders sent from the front of house [6, 7]. MezFlow provides a single-screen, software-only KDS that runs in an ordinary browser and shows every active order as a card that can be filtered by status and by kitchen category. A real-time layer built on the WebSocket protocol, which keeps one connection open so that either side can send messages at any time, pushes every change to every connected screen as it happens [8]. The backend also measures preparation time against an expected time for each item and flags delayed orders automatically.

Restaurants also need tools behind the scenes. A management console lets the manager configure the menu, tables and staff, and role-based access control limits each person to the screens that suit the role [9]. A grounded Artificial Intelligence (AI) feature explains dishes to customers in English and Urdu using only the information supplied by the restaurant, which keeps the explanation from inventing facts [10]. Finally, MezFlow tracks the physical table separately from the digital session, so a table becomes available again only after staff confirm that it has been cleaned and not merely because the bill has been paid.

1.1 Product Purpose

The main goal of MezFlow is to help small and medium restaurants run their dining room and kitchen as one connected workflow instead of a chain of memory, paper and verbal messages. A product purpose states the need that the product answers [11]. The need here is real: 41% of United States restaurant operators funded technology to make the kitchen more productive or efficient in 2023, and 52% planned to do so in 2024 [12]. Established platforms address parts of this need but often depend on dedicated hardware, as the commercial-grade kitchen display hardware listed by Toast and the iPads, printers and KDS touchscreens in the TouchBistro setup show [1, 7]. MezFlow is designed to make the same benefits available through browsers that the restaurant already owns, with no proprietary hardware.

Because good service is more than taking an order, MezFlow follows the whole workflow of Customer, Table Session, Order, Kitchen, Preparation, Serving, Bill, Payment, Table Cleanup and Available. It gives diners a shared, real-time order and a clear explanation of unfamiliar dishes, and it gives the kitchen one organized queue with automatic delay flags [4, 8]. It gives cashiers an itemized bill tied to a verified table session, and it gives managers one console for the menu, the tables, the staff and the operational records. The project follows the Agile Scrum method with two-week sprints and delivers the twelve modules of the system incrementally over a 16-week implementation window [13]. The first review point is the mid-phase presentation in week 7, where the first three chapters of this documentation and the first three modules, about 25% of the functionality, are presented.

Ultimately, MezFlow is not just another ordering page. It is meant to act as a dependable service partner for a restaurant by bringing the diner, the kitchen, the cashier and the manager into one real-time system. Success will be judged against measurable targets: a first page load in under 3 seconds on a typical mobile connection, propagation of order and table changes to every connected client within 2 seconds, correct handling of at least 50 concurrent WebSocket connections and 10 simultaneous table orders, and a System Usability Scale (SUS) score of at least 70 from at least 10 participants [8, 14]. The project is planned to be completed by the end of the second Final Year Project (FYP) semester, FYP-II, which follows FYP-I (Fall 2026), and the restaurant benefits through fewer ordering mistakes, one organized view of the kitchen, automatic detection of delayed orders and a reliable record of which tables are truly free.

1.2 Product Scope

The scope of a product states the boundaries of what the system will and will not do [11]. MezFlow is an all-in-one platform for dine-in service in independent, small and medium restaurants, cafés and food-court outlets with roughly 5 to 30 tables. At its core it provides a secure account system with five roles, a QR-activated digital menu with live pricing and availability, a collaborative table session with a shared cart, a single-screen kitchen display with delay detection, an order lifecycle and a table lifecycle that are kept in step in real time, a billing workflow, a management console, service requests and notifications, analytics and an audit trail. In total the system is organized into twelve modules, and a grounded AI feature explains dishes in English and Urdu using only restaurant-supplied information [10, 8].

Items that are explicitly outside the scope are recorded so that expectations stay realistic within the project timeline [11, 13]. MezFlow does not include food delivery or rider marketplace integration, full accounting, Enterprise Resource Planning (ERP) or payroll systems, inventory and supply-chain management beyond basic availability toggles, custom POS hardware, biometric or facial-recognition identification, native mobile applications, because the customer experience is delivered as a Progressive Web App [4], or automated detection of physical table cleanliness, because cleanup is confirmed by staff.

1.2.1 Existing System Description

When we look at the restaurant technology that is available today, most products do one part of the job well. Some are full POS platforms that handle orders, payments and reports, others are QR menus that mainly collect the order, and in the vendor material reviewed here none documents a connected workflow that joins the diner, the kitchen, the cashier and the table in one real-time system [1, 2, 5]. Ten widely used applications were reviewed. Only features that the vendors document in their public pages are compared, as read on 21 October 2026, except for mr yum, whose site could not be read automatically and whose entry follows the project proposal's review. A feature that is described as not documented may still exist in the product, but it is not described in the cited source. Each review below lists five main features and five limitations that matter for MezFlow.

1.2.1.1 Toast POS & KDS

Toast is a US-based restaurant POS platform that bundles ordering, payment processing and a Kitchen Display System, which its site lists under commercial-grade hardware [1]. It is aimed at full-service and quick-service restaurants.

The main features of Toast include [1, 15]:

- Kitchen Display System for orders from every channel
- Online ordering and handheld POS
- QR-code Mobile Order & Pay at the table
- Tabs that servers and guests can both start and add to
- Toast IQ AI assistant for business data

The limitations of Toast are given below [1, 15]:

- Kitchen display sold as a hardware product
- Live per-diner shared cart not documented
- KDS timers and delay alerts not documented
- Table management and table status not documented
- Dish-explanation AI not documented

1.2.1.2 Square for Restaurants

Square for Restaurants is a POS application with a free plan that is popular with small and medium food businesses because of its low upfront cost and simple setup [2]. Hardware is bought separately and there are no locked-in contracts.

The main features of Square for Restaurants include [2, 16]:

- Free plan with optional KDS and prep insights
- Table and seat management
- QR code ordering mapped to each table
- Automatic order-ready texts

- No locked-in contracts

The limitations of Square for Restaurants are given below [2, 16]:

- Hardware not included with the subscription
- Live shared cart for several diners not documented
- KDS timers and delay alerts not documented
- Table status states not documented
- Dish-explanation AI not documented

1.2.1.3 mr yum

Mr yum is a QR-code ordering and payment platform used by hospitality venues in Australia, the UK and the US, in which customers scan a code at the table, browse the menu in a browser, and order and pay without installing an app [5]. This entry follows the project proposal's review of the vendor site.

The main features of mr yum include [5]:

- Zero-install browser-based QR ordering
- Direct-to-kitchen ticket printing
- Item modifiers and customization
- Integrated at-table payment
- Menu-item performance analytics

The limitations of mr yum are given below [5]:

- One order per scan
- Ticket printers rather than a documented KDS
- Delay detection not documented
- Static menu text with no AI explanation documented
- Table state after payment not documented

1.2.1.4 Lightspeed Restaurant

Lightspeed Restaurant is a cloud POS platform with a customizable menu, a kitchen display, an adjustable floorplan and mobile tableside order-taking [6]. Its site says that it can provide the hardware a restaurant needs, such as printers, iPads and cash registers.

The main features of Lightspeed Restaurant include [6]:

- Kitchen display for orders as they are sent
- Adjustable floorplan
- Order and pay at the table from a phone
- Offline mode
- Staff access limited to what each person needs

The limitations of Lightspeed Restaurant are given below [6]:

- Hardware offered: printers, iPads and cash registers
- QR table ordering not documented (QR appears only for gift cards)
- Shared table order not documented
- Table status states not documented
- Delay alerts and AI not documented

1.2.1.5 TouchBistro

TouchBistro is an iPad-based restaurant POS that provides tableside ordering, customizable floor plans and a Kitchen Display System with color-coded ticket statuses, station routing and bump-bar support [7].

The main features of TouchBistro include [7]:

- Tableside ordering on iPad
- KDS with color-coded ticket statuses
- Station routing and bump bar
- Reports on average tickets and bump time
- Table status from seating until cleared (Reservations)

The limitations of TouchBistro are given below [7]:

- Setup needs iPads, printers, KDS touchscreens and a router
- QR guest ordering not documented
- Shared table cart not documented
- Automatic late-order alerts not documented
- AI features not documented

1.2.1.6 Revel Systems

Revel Systems describes itself as a pioneer of the cloud iPad POS, with kitchen display syncing, dining-room layouts and mobile order takers for tableside ordering [17].

The main features of Revel Systems include [17]:

- Cloud iPad POS
- Immediate sync between table orders and the kitchen display
- Dining-room layout and slow-turning table data
- Mobile order takers for tableside ordering
- Stand with a customer-facing display

The limitations of Revel Systems are given below [17]:

- iPad stands and handheld hardware
- QR menu self-ordering not documented
- Shared table cart not documented
- Delay alerts and preparation-time tracking not documented

- AI features not documented

1.2.1.7 Epos Now

Epos Now is a hospitality POS with table plans, tableside ordering and payment, and a real-time link between front and back of house through kitchen printers or a kitchen display system [18].

The main features of Epos Now include [18]:

- Customisable table plans
- Time-at-table tracking
- Kitchen display with order-time tracking
- Tableside ordering and payment
- QR code self-serve ordering

The limitations of Epos Now are given below [18]:

- Hardware offer tied to a 12-month subscription
- Shared table cart not documented
- Table status states not documented
- Delay alerts not documented
- AI features not documented

1.2.1.8 Flipdish

Flipdish is an online and table ordering platform in which a table QR code pre-fills the table number and guests order through the mobile version of the restaurant's website [19].

The main features of Flipdish include [19]:

- Table QR codes that pre-fill the table number
- Ordering through the mobile website
- Card payment in advance or cash
- Table number passed to the POS and kitchen screens
- Orders sent into the existing POS

The limitations of Flipdish are given below [19]:

- Setup through customer success or a sales manager
- Open tabs not supported, so guests pay in advance
- Shared basket for several guests not documented
- Table status tracking not documented
- Own KDS, delay alerts and AI not documented

1.2.1.9 Menu Tiger

Menu Tiger is a QR-code menu and ordering product in which restaurants receive orders from customers who scan a QR code, with order tracking statuses Pending, Preparing and Ready [20].

The main features of Menu Tiger include [20]:

- QR-code digital menu
- Orders from QR scans
- Contactless payments through the restaurant's Stripe account
- Menus in multiple languages
- Sales and menu analytics

The limitations of Menu Tiger are given below [20]:

- Kitchen display not documented
- Table management not documented
- Staff roles not documented
- Shared cart for several diners not documented
- Delay detection and AI dish explanation not documented

1.2.1.10 Oracle MICROS Simphony

Oracle MICROS Simphony is a cloud POS platform used by global enterprise chains, and it integrates with Oracle Hospitality Kitchen Display Systems, which monitor order timing and alert managers and kitchen staff to high ticket times [21, 22].

The main features of Oracle MICROS Simphony include [21, 22]:

- Cloud POS for enterprise chains
- Real-time table management
- KDS with order timing and high-ticket-time alerts
- Station routing and expediter view
- Menu Item Preparation Report

The limitations of Oracle MICROS Simphony are given below [21, 22]:

- Workstation, tablet and touchscreen hardware
- QR guest ordering not documented
- Shared table cart not documented
- Table cleanup state not documented
- AI dish explanation not documented

Table 1.1 compares the ten reviewed applications with MezFlow. A tick means that the feature is documented in the reviewed vendor source, and a cross means that it is not documented in that source; it does not prove that the product lacks the feature [1, 2, 5, 6, 7, 17, 18, 19, 20, 21].

Table 1.1: Applications Comparison

Features	Toast	Square	mr yum	Lightspeed	TouchBistro	Revel	Epos Now	Flipdish	Menu Tiger	Oracle MICROS	MezFlow
QR or phone table ordering	✓	✓	✓	✓	✗	✗	✓	✓	✓	✗	✓
Live shared cart with per-diner attribution	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Kitchen Display System	✓	✓	✗	✓	✓	✓	✓	✗	✗	✓	✓
Software-only KDS on any browser device	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Live order status for the diner	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓
Preparation-time tracking	✗	✗	✗	✗	✓	✗	✓	✗	✗	✓	✓
Automatic delayed-order alerts	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓
Table and floor management	✗	✓	✗	✓	✓	✓	✓	✗	✗	✓	✓
Table tracked until cleaned after payment	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✓
Grounded AI dish explanation	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓

Key: ✓ = documented in the reviewed vendor source; ✗ = not documented in the reviewed source (the product may still offer it); the MezFlow column shows the planned features. The mr yum column follows the project proposal.

In the reviewed sources, QR ordering, POS and kitchen display functions are well covered. The features that none of the sources documents are a live shared cart with per-diner attribution, a software-only kitchen display that runs on any browser, live order status for the diner, and grounded AI dish explanations, and a table tracked until it is cleaned is documented only for TouchBistro. These gaps define the value of MezFlow [1, 2, 7, 22].

1.2.2 Future System Usage Analysis

MezFlow is designed with scalability and future adaptability in mind. Scalability is the ability of a system to handle growth in users, data and functions without a redesign [23]. The data of every restaurant is kept separate, so the platform can later serve more locations, languages, currencies and payment methods, including an online payment gateway and the multi-site control that enterprise platforms such as Oracle MICROS Simphony already offer [21]. A restaurant group could manage the menus, staff and reports of all its branches from one console, while each branch keeps its own tables and kitchen screen.

Advanced AI could refine the platform further. Toast already offers an AI assistant, Toast IQ, that connects data and workflows across the business [1], and MezFlow could in the future learn expected preparation times from its own order history, suggest dishes that match a diner's stated preferences or allergies, and forecast busy periods so that the kitchen can plan. The existing dish-explanation feature would remain grounded in restaurant-supplied data, because retrieval from trusted sources is what keeps generated text accurate [10].

Further technology could be added as the platform matures. Because every state change already travels as a WebSocket event, the same channel could carry readings from table-occupancy or cleanliness sensors, so that the Awaiting Cleanup state could be confirmed or prompted automatically and not only by staff [8]. Kitchen sensors could feed temperature and timing data into delay detection. Native mobile applications, offline operation for the customer app, voice ordering and richer notifications could also follow once the core platform is proven [4, 24].

Finally, integrations could extend the reach of the system. Food delivery and rider marketplace connections, inventory and supplier management, accounting exports, loyalty programs and reservations are offered by some existing products [6, 18]. Adding them later would let MezFlow grow from a dining-room and kitchen platform into a full restaurant management suite, while the open-source stack and the REST interface keep these extensions practical and affordable [25, 26].

1.3 Objectives

The core objective of MezFlow is to design, build and test an integrated, real-time platform that connects customer ordering, kitchen operations, billing and restaurant management for small and medium dine-in restaurants. Objectives are most useful when they are specific and measurable [27], so every objective below carries a target. The customer side is a zero-install, QR-activated Progressive Web App through which diners browse the menu and order, with a first-load time under 3 seconds on a typical mobile connection [4]. Multiple diners who scan the same QR code share one live order, and the backend records which diner added each item. A real-time layer built on WebSockets propagates order, kitchen and table-status changes to every connected client within 2 seconds under normal network conditions [8].

On the kitchen side, MezFlow develops a single-screen Kitchen Display System that shows every active order as an independent card and supports category and status filtering, and it is targeted to display at least 10 simultaneous table orders submitted within a 2-minute window correctly and without interference [7, 6]. Preparation-time tracking compares the elapsed time against a configurable expected time per item and flags delayed orders automatically, which goes beyond the order-timing and bump-time reports that existing systems document [7, 22]. The order lifecycle (New, Confirmed, Preparing, Ready, Dispatched) and the table lifecycle (Available, Occupied, Awaiting Cleanup, Available) are implemented as two independent, auditable state machines, so that payment closes the digital session without assuming that the table is physically free [28].

The remaining objectives cover billing, intelligence and administration. A digital billing workflow lets the customer request the bill and an authorized staff member process payment [1, 2]. A grounded “Explain This Dish” feature generates a natural-language explanation strictly from restaurant-provided structured data, without inventing facts that the restaurant did not supply [10]. A management console for menu, table, staff and pricing configuration is protected by role-based access control for the customer, kitchen staff, service staff, cashier and manager roles [9, 29]. Finally, the platform is evaluated through load testing with at least

50 concurrent WebSocket connections without state corruption and through a usability trial with at least 10 participants, targeting a System Usability Scale score of at least 70 [8, 14]. Through these objectives MezFlow aims to become a reliable and affordable operating system for the dining room and the kitchen.

1.4 Problem Statement / Limitations

A problem statement describes the present situation, the gap between that situation and the desired one, and the people affected by it [11]. In many small and medium restaurants, ordering, kitchen coordination, menu management and table turnover still run on partially manual workflows, and customers often cannot get a clear explanation of an unfamiliar dish without a waiter. Modifiers communicated verbally, such as spice level, removed ingredients or portion changes, are easy to mishear or forget between the table and the kitchen. When several people share a table there is usually no single shared order: either one person orders for everyone, or each diner must call a waiter separately. Even products that allow guests to order from their phones, such as Toast Mobile Order & Pay and Square QR ordering, describe a tab that servers and guests can both add to, or one consolidated ticket rather than a live shared cart that each diner builds [15, 16].

The kitchen faces a second gap. Paper tickets or a printer queue give the kitchen no organized, filterable view of every active order, and the kitchen display systems that do exist usually depend on vendor hardware or a vendor ecosystem [1, 7, 2]. Automatic alerts for delayed orders are documented for enterprise platforms such as Oracle MICROS but are not described for most small-restaurant products [22]. A third gap concerns the table itself. Most systems treat a paid bill as the end of the workflow, so nothing distinguishes a table that is paid but still occupied from one that is clean and ready, and only TouchBistro documents table status from seating until the table is cleared [7]. Restaurant operators feel the pressure behind these gaps, since 47% of United States operators expect technology and automation to become more common in their segment to help with labor shortages [12].

The users affected are the restaurant owners and managers, who need fewer mistakes and visibility into which orders and tables need attention, the kitchen staff, who need one organized queue instead of scattered tickets, the waiters, who need fewer repetitive trips, the cashiers, who need an accurate itemized bill, and the customers, who want to order at their own pace and track their food [11]. The core problem is therefore not the absence of digital tools but the absence of one connected, real-time workflow, and MezFlow aims to close this gap with a single platform that combines shared table ordering, a software-only kitchen display, delay detection, an explicit table lifecycle and grounded dish explanations.

To keep the project achievable within the Final Year Project timeline, a number of limitations apply to the system [11]. The pilot targets dine-in service in restaurants with roughly 5 to 30 tables, so delivery and rider marketplace integration, which products such as Toast and Lightspeed offer, are not included [1, 6]. Payment is recorded by a cashier as cash or card and an online payment gateway is not part of the core scope, inventory is limited to basic availability toggles, and accounting, payroll and Enterprise Resource Planning (ERP) features

are excluded. The customer experience is delivered as a PWA and not as a native mobile application [4], and table cleanliness is confirmed by staff and is not sensed automatically. The system needs a working internet connection, and unstable restaurant Wi-Fi is handled by automatic reconnection and state resynchronization and not by full offline operation [8, 24]. AI dish explanations are limited to the structured fields supplied by the restaurant and are reviewed before they are published [10].

1.5 Proposed Solution

A solution architecture describes the main components of a system and how they communicate [25]. MezFlow is an all-in-one platform for running the dining room and the kitchen of a restaurant, built from a customer Progressive Web App, a Kitchen Display System web application, a management console web application and a backend Representational State Transfer (REST) and WebSocket Application Programming Interface (API) with a PostgreSQL database [25, 8, 30]. It brings together what most restaurant tools keep apart. The workflow it supports is Customer, Table Session, Order, Kitchen, Preparation, Serving, Bill, Payment, Table Cleanup and Available.

For the diner, the Digital Menu and QR Table Access module gives each table its own QR code that opens the menu with categories, descriptions, images, prices, live availability and structured modifiers directly in the browser [3, 4]. The Collaborative Table Ordering module lets everybody at the table join one shared session, add items to one live cart with per-diner attribution and submit a single order for the table. The AI-Assisted Dish Explanation module adds an “Explain This Dish” button that describes a dish in English or Urdu using only the ingredients, flavor, texture, heat level and portion size that the manager has supplied, so the answer stays grounded in the restaurant’s own data [10].

For the kitchen, the Kitchen Display System module shows every active order as an independent card with its order number, table number, items, modifiers, elapsed time and status, and it can be filtered by status and by kitchen category such as Grill, Fry, Hot Kitchen and Beverage [7, 6]. The Preparation-Time Tracking and Delay Detection module stores an expected preparation time for every item, measures the elapsed time from the moment preparation starts and flags an order as delayed when the expected time is exceeded [22].

A state machine describes an entity as a set of states with the transitions allowed between them [28]. The Order Lifecycle and Real-Time Synchronization module models the order as New, Confirmed, Preparing, Ready and Dispatched and pushes every change over WebSockets to the customer app, the kitchen display and the management console [8]. The Table Lifecycle Management module models the table separately as Available, Occupied, Awaiting Cleanup and Available again. The Billing and Payment Workflow module lets the customer request the bill and lets the cashier process payment, and the backend recomputes every price from the menu and never trusts a price sent by the customer device. Payment closes the digital session but never frees the table, which becomes available only after staff confirm cleanup [1, 2].

Behind the scenes, the Identity and Access Control module provides sign-up and login with one-time password verification and JSON Web Token sessions for five roles, namely customer, kitchen staff, service staff, cashier and manager [9, 29]. The Management Console module lets the manager maintain the menu, prices, preparation times, tables, QR codes, staff and restaurant settings. The Service Requests and Notifications module lets customers call a waiter or ask for water and informs them about order and bill status [24], and the Analytics and Operational Records module offers dashboards of orders, average preparation time and delayed-order rate together with a full audit trail and exportable reports.

The platform is built entirely on open-source frameworks and standard web protocols: React/Next.js and Tailwind CSS for the interfaces, Python/FastAPI for the backend, PostgreSQL for the data and WebSockets for real-time communication [31, 26, 30, 8]. All cart, order, table and price logic is held on the server, which acts as the single source of truth, and the system separates the data of different restaurants so that it can later serve more than one location [25]. MezFlow therefore works like a service desk, a kitchen board and a cashier assistant in one system that runs on the devices a restaurant already owns.

1.6 Intended Market of Product

MezFlow is made for independent, small and medium dine-in restaurants, cafés and food-court outlets with roughly 5 to 30 tables, where staff currently rely on verbal or paper-based ordering. A target market is the group of customers whose needs a product is designed to meet [32]. This segment is poorly served by existing products: platforms such as Toast list commercial-grade kitchen hardware, TouchBistro’s setup requires iPads, printers, touchscreens and a supported router, and lighter tools such as mr yum and Menu Tiger document QR ordering and order statuses or kitchen tickets rather than a connected table and kitchen workflow [1, 7, 5, 20].

It is also a good fit for independent owners and small chains who want to modernize without a large investment. The worth of a product to its market is the customer value it delivers relative to the cost the customer bears [32]. Because MezFlow needs no proprietary hardware and is built entirely on open-source frameworks and standard web protocols, it can be self-hosted at low cost, and the kitchen display runs on any browser-capable tablet, laptop or low-cost computer that the restaurant already owns [31, 26, 30, 4].

As more restaurants invest in technology, the platform also suits operators who want to speed up service and reduce staffing pressure. In the United States, 47% of operators expect technology and automation to become more common in their segment to help with labor shortages, and 52% planned to invest in making the kitchen more productive or efficient in 2024 [12]. MezFlow’s single kitchen screen, automatic delay flags and shared table cart address exactly these priorities.

Diners are part of the market as well. In the same survey, 57% of limited-service customers and 59% of full-service customers said that they would access a menu on a smartphone using a QR code [12]. For the tech-savvy crowd, the real-time order tracking and the AI dish

explanation in English and Urdu make the experience especially appealing, and for groups who share a table the shared cart removes the need to pass one person's phone around.

In the longer term the multi-tenant design allows restaurant groups with several branches to use the same platform, in the way that enterprise products serve multi-site chains [21]. The market figures above come from United States surveys, so local demand will be examined during the requirements phase through interviews with restaurant staff in Pakistan [12, 13]. In short, MezFlow is built for any dine-in restaurant that cares about faster service, fewer mistakes and a clear view of its dining room and kitchen in one easy, modern platform.

1.7 Intended Users of Product

A user class is a group of people who interact with a system in a similar way and with similar needs, and access control assigns each class a role that limits what it can see and do [9, 11]. MezFlow serves a wide variety of people in a restaurant, and its five roles are the customer, the kitchen staff, the service staff, the cashier and the manager. Customers, from solo diners to large groups, scan the table QR code, browse the menu, ask for dish explanations, build a shared cart, follow their order live, request the bill and call for service, all without installing an application [4]. Kitchen staff use the single-screen kitchen display to see every active order in one organized queue, filter by status and station, advance each order and watch for delayed orders [7].

Service staff respond to service requests, watch the live table-status view and confirm that a table has been cleaned, while cashiers review bill requests, process cash or card payment and close the session. Managers configure the menu, prices, tables, staff and restaurant settings and review the analytics and the audit trail. Restaurant owners are also stakeholders, since they benefit indirectly through lower order-taking overhead, fewer mistakes and better visibility into restaurant operations [11]. Staff are expected to need only brief on-site training because the kitchen and service interfaces are simple and filter-driven, and the usability of these interfaces is evaluated with the System Usability Scale [14].

1.8 Software Process Model

The MezFlow team follows the Agile Scrum process model, working in two-week sprints on a prioritized product backlog of the twelve modules, and every sprint ends with a working increment that is reviewed with the supervisor [13]. Figure 1.1 summarizes the cycle, and the subsections below introduce the model, justify the choice and describe the steps.

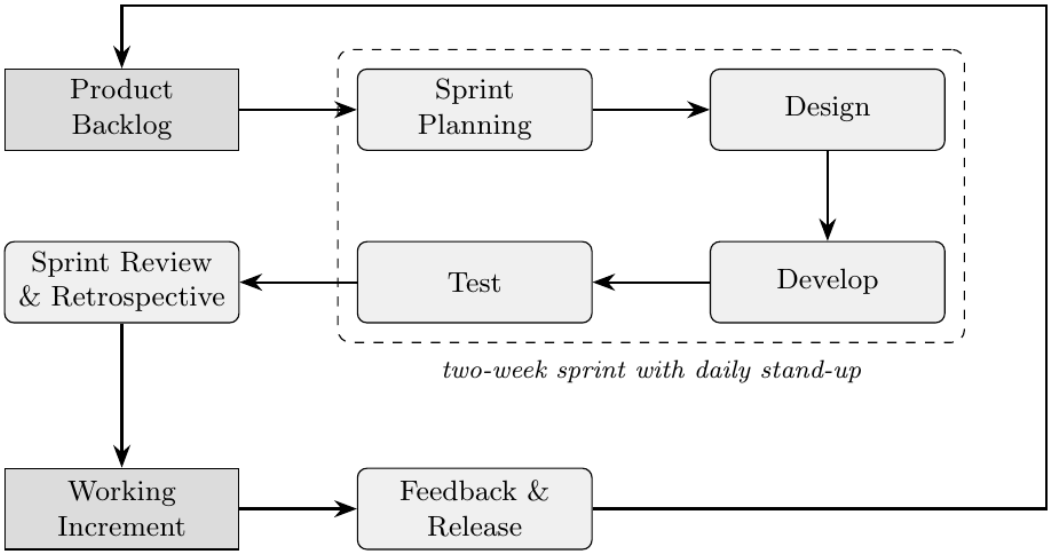


Figure 1.1: Agile process model

1.8.1 Process Model Introduction

A software development process model is the organized set of activities by which software is specified, built, tested and delivered, and it serves as the blueprint that guides the team from the initial idea to the final release [23]. The choice of model is determined by the unique requirements of the project, such as its scope, timeline, budget and team size, and a structured model keeps the development work organized, efficient and targeted [23].

Scrum is a lightweight framework that helps teams generate value through adaptive solutions for complex problems [13]. Work is organized into Sprints, which are fixed-length events of one month or less. The Product Backlog is the ordered list of what is needed to improve the product, each Sprint contains Sprint Planning, Daily Scrums, a Sprint Review and a Sprint Retrospective, and each Sprint produces an Increment that meets the team's definition of done [13]. Scrum is an instance of Agile development, which values responding to change over following a plan and delivers working software frequently, from a couple of weeks to a couple of months [33].

For the proposed system, the team has chosen Agile Scrum. The three team members act as the Scrum team, which Scrum describes as typically ten or fewer people [13], and the project supervisor acts as the stakeholder who attends each Sprint Review. The project scope is twelve modules, the timeline is a 16-week implementation window across two semesters, and the budget is limited to free and open-source tools. The practices used are two-week sprints, a prioritized product backlog, short stand-ups, a sprint review with the supervisor and a retrospective, and the supporting tools are Git and GitHub for version control, GitHub Actions for continuous integration, and pytest, Postman and Locust for testing.

1.8.2 Justification of Proposing the Process Model

Agile Scrum is the recommended process model for MezFlow because the requirements of kitchen staff, cashiers and diners will become clearer as working software is shown to them,

and a process that expects change handles this better than a fixed plan [23, 33]. Two parts of the system carry genuine uncertainty, namely how the shared table session behaves under concurrent use and how kitchen staff want to filter and act on the order queue during a rush, and each sprint review gives feedback on exactly these points [13]. By delivering functional software in increments, the most essential modules are built first, in an order that puts the minimum viable product first, so core features are available early and the team can refine the later modules in the light of what it learns. Fixed two-week sprints also help to control scope creep across twelve modules within the Final Year Project timeline, because no change is made during a sprint that would endanger the sprint goal, and a three-member team can coordinate easily through short stand-ups and a shared backlog [13].

1.8.3 Steps of Process Model

The Agile Scrum lifecycle for this project emphasizes continuous alignment with user expectations through the following structured phases [13].

Project Planning: Defining the scope, establishing timelines and allocating work among the team members. Requirements are gathered through a review of existing POS and QR-ordering products, informal interviews with restaurant staff and usability walkthroughs with prospective diners, and they are written as user stories in a product backlog for all twelve modules. The first milestone is the mid-phase review in week 7, where the first three chapters of the documentation, the five-application comparison and the first three modules, Identity and Access Control, Digital Menu and QR Table Access, and Collaborative Table Ordering, which is about 25% of the functionality, are presented [13, 11].

Iterative Development (Sprints): Collaboratively designing, coding and testing the selected features in time-boxed iterations of two weeks. At the start of every sprint the team selects the highest-priority backlog items and assigns owners, and automated tests run on every code change. The modules are allocated to the two semesters as follows [13, 33]. In FYP-I the team builds the foundation and the ordering path: requirements and design, Modules 1 to 4, and the groundwork of Module 8, Billing and Payment Workflow, which is its data model for bills and payments and its server-side price validation, together with unit tests of the price-validation logic. In FYP-II the team completes Module 8, which is the bill request, cashier payment and session closure flow, and adds Modules 5 to 7 and 9 to 12. Module 8 is therefore completed in FYP-II. Its testing milestone is an integration test of the whole sequence of bill request, payment, session closed and table moved to Awaiting Cleanup, which can run only after Module 5 (Order Lifecycle and Real-Time Synchronization) and Module 9 (Table Lifecycle Management) exist, so it is scheduled in FYP-II before the load testing and the usability trial.

Continuous Feedback Loop: Actively engaging the supervisor, restaurant staff and prospective diners to review each incremental release, prioritize the upcoming functionality and ensure that the product meets user needs. Every sprint ends with a sprint review and a retrospective whose outcomes flow back into the product backlog, and after the final sprint the integrated system is evaluated through load testing and a usability trial using the System Usability Scale before the documentation is finalized [13, 14, 8].

References

[1] Toast, Inc. Toast POS and Kitchen Display System [Online]. Available: https://pos.toasttab.com. Last Accessed on October 21, 2026.

[2] Block, Inc. Square for Restaurants [Online]. Available: https://squareup.com/us/en/point-of-sale/restaurants. Last Accessed on October 21, 2026.

[3] ISO/IEC 18004:2015, *Information Technology — Automatic Identification and Data Capture Techniques — QR Code Bar Code Symbology Specification*, 2015.

[4] MDN Web Docs. Progressive Web Apps — Best Practices [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. Last Accessed on September 25, 2026.

[5] Mr Yum Pty Ltd. Mr Yum — QR Ordering and Payments [Online]. Available: https://mryum.com. Last Accessed on September 25, 2026.

[6] Lightspeed Commerce Inc. Lightspeed Restaurant Features [Online]. Available: https://www.lightspeedhq.com/products/restaurant/features/. Last Accessed on October 21, 2026.

[7] TouchBistro Inc. TouchBistro U.S. Product Guide 2026 [Online]. Available: https://www.touchbistro.com/wp-content/uploads/2024/09/US-TouchBistro-Product-Guide-2026.pdf. Last Accessed on October 21, 2026.

[8] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF, RFC 6455, Dec. 2011 [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455. Last Accessed on September 25, 2026.

[9] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[10] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[11] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, 2018.

[12] National Restaurant Association. Restaurant Technology Landscape Report 2024 [Online]. Available: https://go.restaurant.org/rs/078-ZLA-461/images/NatRestAssoc_TechLandscapeReport_2024.pdf. Last Accessed on October 21, 2026.

[13] K. Schwaber and J. Sutherland. (2020, Nov.). The Scrum Guide [Online]. Available: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf. Last Accessed on October 21, 2026.

[14] J. Brooke, "SUS: A quick and dirty usability scale," in *Usability Evaluation in Industry*, P. W. Jordan *et al.*, Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.

[15] Toast, Inc. Toast Mobile Order & Pay [Online]. Available: https://pos.toasttab.com/ca/products/mobile-order-and-pay. Last Accessed on October 21, 2026.

[16] Block, Inc. Square QR Code Ordering [Online]. Available: https://squareup.com/us/en/online-ordering/qr-code-ordering. Last Accessed on October 21, 2026.

[17] Revel Systems. Restaurant POS System [Online]. Available: https://revelsystems.com/pos-systems/restaurant-pos. Last Accessed on October 21, 2026.

[18] Epos Now. Hospitality POS System [Online]. Available: https://www.eposnow.com/au/systems/hospitality-pos/. Last Accessed on October 21, 2026.

[19] Flipdish. Table Ordering and QR Code FAQ [Online]. Available: https://help.flipdish.com/en/articles/9585439-table-ordering-and-qr-code-faq. Last Accessed on October 21, 2026.

[20] Menu Tiger. Features [Online]. Available: https://www.menutiger.com/features. Last Accessed on October 21, 2026.

[21] Oracle. Oracle MICROS Simphony POS [Online]. Available: https://www.oracle.com/food-beverage/restaurant-pos-systems/simphony-pos/. Last Accessed on October 21, 2026.

[22] Oracle. Oracle MICROS Kitchen Display Systems (Data Sheet) [Online]. Available: https://oracle.com/a/ocom/docs/industries/hospitality/oracle-micros-kitchen-display-systems-ds.pdf. Last Accessed on October 21, 2026.

[23] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[24] MDN Web Docs. Offline and Background Operation [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation. Last Accessed on September 25, 2026.

[25] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[26] S. Ramírez. FastAPI [Online]. Available: https://fastapi.tiangolo.com. Last Accessed on October 21, 2026.

[27] G. T. Doran, "There's a S.M.A.R.T. way to write management's goals and objectives," *Manage. Rev.*, vol. 70, no. 11, pp. 35–36, Nov. 1981.

[28] D. Harel, "Statecharts: A visual formalism for complex systems," *Sci. Comput. Program.*, vol. 8, no. 3, pp. 231–274, Jun. 1987.

[29] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015 [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519. Last Accessed on October 21, 2026.

[30] The PostgreSQL Global Development Group. About PostgreSQL [Online]. Available: https://www.postgresql.org/about/. Last Accessed on October 21, 2026.

[31] Vercel, Inc. Next.js Documentation [Online]. Available: https://nextjs.org/docs. Last Accessed on October 21, 2026.

[32] P. Kotler and K. L. Keller, *Marketing Management*, 15th ed. Harlow, U.K.: Pearson, 2016.

[33] K. Beck *et al.* (2001). Manifesto for Agile Software Development [Online]. Available: https://agilemanifesto.org. Last Accessed on October 21, 2026.